Előadói jegyzet - ConfigMap, Secret, External Secrets Operator

---

1. rész: ConfigMap

Slide: Mi az a ConfigMap?

- A ConfigMap a Kubernetes natív .env fájlja: kulcs-érték párokat tárol nem érzékeny konfigurációhoz.
- Namespace-scoped: egy namespace-en belül egyedi a név, különböző namespace-ekben lehet ugyanaz a név.
- Az 1 MiB méretkorlát az etcd hard limitje miatt van - nem véletlen szám.
- Sokan azzal kezdenek, hogy titkokat is ide rakjanak - épp ezért lesz a következő szekció a Secret-ről.

Slide: Milyen problémát old meg?

- Ez a 12-factor app III. alapelvének gyakorlati megvalósítása K8s-ben: válaszd szét a kódot a konfigurációtól.
- "Build once, run anywhere" - egy image, sok környezet.
- Konkrét példa: ugyanaz az image fut dev-en `LOG_LEVEL=debug`-gel, prod-on `LOG_LEVEL=info`-val, újraépítés nélkül.

Slide: Előnyök / Hátrányok

- A "Hátrányok" oszlop a fontosabb: ami **nem** való ConfigMap-be.
- Érzékeny adat - jelszó, API token, TLS kulcs - soha nem kerülhet ConfigMap-be. Ez vezet át a Secret szekcióra.
- A gyakorlatban majdnem minden csapatban előfordult, hogy valaki jelszót rakott ConfigMap-be - normális, ezért érdemes beszélni róla.

Slide: Hogyan működik belül?

- A két csatolási mód a legfontosabb koncepció: **env változó** vs **volume mount**.
- Env változó: pod indulásakor egyszer beolvasódik, utána statikus.
- Volume mount: folyamatosan frissül, a kubelet ~30-60 másodpercenként szinkronizálja (`syncFrequency` beállítás).

Slide: ConfigMap létrehozása

- A CLI parancs a háttérben YAML-t generál - a `-o yaml --dry-run=client` kapcsolóval kiíratható.
- A gyakorlatban GitOps miatt a YAML-os létrehozás dominál; a CLI inkább ad-hoc vagy dev célra való.

Slide: Pod-hoz csatolás - env változó

- Az `envFrom` az összes kulcsot env változóként kihúzza a pod-ba.
- Ha csak egy konkrét kulcs kell, akkor `env: –name: –valueFrom: configMapKeyRef:` szintaxis használható.
- A figyelmeztetés a kulcs: a pod nem látja az új értékeket módosítás után, **restart kell** hozzá.

Slide: Pod-hoz csatolás - volume mount

- Volume mount esetén minden kulcs **egy fájlként** jelenik meg a `mountPath` alatt - pl. `/etc/config/LOG_LEVEL`.
- A `subPath` használata kikapcsolja a hot reload-ot - kevesen tudják, gyakori bug forrása.
- A pod-on belül `ls /etc/config` és `cat /etc/config/LOG_LEVEL` parancsokkal látható a tartalom (a demóban végigmegyünk).

Slide: Csatolási módok összehasonlítása

- A táblázat tömör összefoglaló a négy csatolási módról.
- Egy mondatban: hot reload csak volume mount esetén működik, és csak akkor, ha **nincs** subPath.

Slide: Biztonság és limitációk

- A legfontosabb pont: a ConfigMap az etcd-ben **plaintext-ben** tárolódik. Ez ugyanígy igaz a Secret-re is.
- Bárki, akinek `kubectl get configmap` joga van, olvashatja a tartalmat - tehát az RBAC a védelem, nem maga a ConfigMap.
- Az immutable flag (K8s 1.19+) performancia optimalizáció: ha tudjuk, hogy nem fog változni, a kubelet nem watcheli folyamatosan.

Slide: Alternatívák

- Helm = template engine, Kustomize = patch overlay - ennyi különbség elég ezen a szinten.
- Spring Cloud Config: app-szintű konfiguráció-szolgáltató, K8s-független.
- CRD-alapú konfig: ha a saját operator-od saját Custom Resource-okat használ konfigként.

Slide: DEMO 1

- Fő üzenet: az env változós pod **nem frissül**, a volume mount-os pod **frissül** restart nélkül.
- A demo során a YAML-t nem soronként magyarázzuk - a `kubectl apply` és `kubectl exec` parancsok a lényeg.
- Ha jut idő, a `kubectl describe pod` kimenetén látható, hogyan jelenik meg a volume mount.

Időzítés - 1. rész

- ~10-12 perc az egész szekció (3 perc demo + 7-9 perc dia).
- Ha csúszunk: a "Hogyan működik belül?" és "Alternatívák" diákat lehet röviden átfutni.
- Ha gyorsak vagyunk: bemutatható a `kubectl edit configmap` live.

---

2. rész: Secret

Slide: Mi az a Secret?

- A Secret a ConfigMap testvére, de érzékeny adatokra szánva.
- Az `Opaque` a default típus - a gyakorlatban 99%-ban ezt használjuk.
- A többi típus (`tls`, `dockerconfigjson`, `service-account-token`) speciális célokra való: a tooling tudja, hogy mit vár tőlük.

Slide: Milyen problémát old meg?

- Történelmileg régen ConfigMap-be kerültek a jelszók is, mert csak az volt elérhető.
- A Secret nem azért biztonságosabb, mert titkosított, hanem mert **a tooling máshogy bánik vele**: tmpfs mount, RBAC, audit, külön típus.

Slide: ConfigMap vs Secret — mikor mit?

- A táblázat döntési segéd a két típus között.
- Hüvelykujj-szabály: ha az adat kiszivárogna és bajt okozna → Secret. Ha nem → ConfigMap.
- Másik vezető kérdés: ha ezt az értéket publikusan kiraknánk a GitHub-ra, baj lenne? Ha igen → Secret.
- Tipikus csapda: a környezet neve (dev/prod) **ConfigMap**-be való, nem Secret-be - mert nem érzékeny adat.

Slide: Valós életbeli példák

- Konkrét példák, hogy mit szoktunk Secret-ben tárolni a gyakorlatban.
- Stripe API kulcs: közvetlen pénzügyi kockázat, ha kiszivárog.
- TLS privát kulcs: ha kiszivárog, a HTTPS forgalom dekódolható.
- JWT aláíró kulcs: ezzel bárki tud "saját maga által hitelesnek tűnő" tokent generálni.
- Közös bennük: **a kiszivárgás közvetlen biztonsági kockázatot jelent** - ez a Secret definíciója a gyakorlatban.

Slide: Előnyök / Hátrányok

- A natív Secret kis projekteknél, dev környezetben elég.
- Éles környezetben önmagában kevés - ha 5 csapat, 3 cluster, GDPR audit jön, már nem.
- Ez vezet át a 3. szekcióhoz: az ESO + külső secret manager.

Slide: A nagy félreértés - base64 ≠ titkosítás

- Ez a szekció legfontosabb diája.
- A base64 csak **kódolás**, NEM titkosítás. Bárki visszafejti egy `base64 -d` paranccsal.
- A demo során ezt élőben végigjátsszuk.
- A "Secret-ek base64-be vannak" kommunikáció félrevezető - ez nem védelem.

Slide: Hogyan működik belül?

- Két nem nyilvánvaló dolog:
  1. Az etcd alapból **NEM titkosítja** a Secret-et - külön be kell kapcsolni az encryption at rest-et.
  2. A pod-on **tmpfs**-en (memóriában, nem disken) mountolódik - kis biztonsági plusz.
- A csatolási módok ugyanazok mint ConfigMap-nél: env változó vagy volume mount.

Slide: Secret létrehozása

- A YAML példa **base64-kódolt** értéket vár - ezt sokan elfelejtik és plaintext-et írnak be helyette.
- Tipp: `echo -n "jelszó" | base64`. A `-n` fontos, nélküle plusz `\n` karakter kerül a kódolt érték végére.
- A `kubectl create secret` parancs ezt automatikusan kódolja.

Slide: Secret típusok

- A táblázat referencia a 6 beépített Secret típusról.
- A `kubernetes.io/tls` kötött struktúra: pontosan `tls.crt` és `tls.key` kulcsokat vár.
- A `dockerconfigjson` az `imagePullSecrets`-hez kell, privát Docker registry esetén.

Slide: Biztonság és limitációk

- Ez a dia készíti elő a 3. szekciót - minden pont egy hiányosság, amit az ESO + Vault megold.
- A legdrámaibb az audit hiánya: nem tudjuk, ki és mikor olvasta a Secret-et.
- Az encryption at rest **nem oldja meg** a `kubectl get secret` problémát - csak a disken titkosít, a kubectl-en át továbbra is plaintext.

Slide: Alternatívák a natív Secret-en túl

- Sealed Secrets: encrypted YAML-t Git-be tehetsz, a klaszter visszafejti egy controllerrel.
- SOPS: általánosabb, fájl-szintű titkosítás (nem K8s-specifikus, akármilyen YAML-ra/JSON-ra használható).
- git-crypt: egyszerű, transzparens Git fájl-titkosítás.
- External Secrets Operator: erre épül a következő rész.

Slide: DEMO 2

- Fő üzenet: `base64 -d` egyetlen parancs, és minden titok plaintext-ben van.
- A `kubectl auth can-i` rész fontos: **a védelem RBAC-ben van, nem a Secret-ben**.
- 2-3 perc bőven elég ennek bemutatására.

Időzítés - 2. rész

- ~10 perc szekció (2 perc demo + 8 perc dia).
- Ha csúszunk: a "Secret típusok" táblázatot lehet átugrani.
- A "base64 ≠ titkosítás" diát **soha nem ugorjuk át** - ez a fő tanulság.

---

3. rész: External Secrets Operator + Vault

Slide: Mi az a külső secret manager?

- Egy dedikált, központi rendszer titkos adatok tárolására, hozzáférés-vezérlésére, naplózására és rotációjára.
- Cloud provider felé: AWS-en valószínűleg Secrets Manager, multi-cloud esetén Vault.
- A demónkban Vault szerepel, de az integrációs logika mindegyiknél ugyanaz.

Felhasználási esetek

- A dinamikus credentialek pont érdekes: a Vault tud időre szóló, automatikusan lejáró DB usereket generálni.
- Hátrányok oldalon: lokális dev-re overkill, nincs értelme.
- Üzleti döntés: van-e csapat, aki üzemelteti? - A Vault nem self-running.

Slide: Két különálló réteg

- A tár (pl. Vault) és az integráció (pl. ESO) **két különböző döntés** - sokan keverik a kettőt.
- A demónk Vault + ESO párost használ, de bármelyik réteg cserélhető (pl. AWS Secrets Manager + ESO, vagy Vault + CSI Driver).

Slide: Mi az az External Secrets Operator?

- Az "Operator" K8s pattern: CRD + reconciliation loop a klaszterben futva.
- Két fő CRD: `SecretStore` (kapcsolat a tárhoz) és `ExternalSecret` (mit, honnan, hova szinkronizálni).
- Transzparencia: a pod nem tud Vault-ról, csak egy sima K8s Secret-et lát.

Slide: SecretStore példa

- A `kubernetes` auth method oldja meg a secret zero problémát - a `serviceAccountRef` adja a JWT-t a Vault-nak.
- A Vault ezt a JWT-t a klaszter API server-én keresztül validálja.

Slide: ExternalSecret példa

- A `refreshInterval: "15s"` csak a demo miatt rövid - prod-ban inkább `5m` vagy `15m`.
- A `data:` szekció **átnevezést** is enged: a Vault-ban `db-password` néven lévő érték a K8s Secret-ben `password` néven jelenik meg.

Slide: Architektúra

- A diagram végigköveti a folyamatot:
    1. Vault-ban él a titok titkosítva.
    2. Az ESO az API-n át lekéri.
    3. K8s Secret-et hoz létre vagy frissít belőle.
    4. A pod ezt a Secret-et mountolja.
- A pod nem tud Vault-ról - ez biztonsági érv is, mert a hibafelület kisebb.

Slide: A "Secret zero" probléma

- A klasszikus filozófiai kérdés: ha minden titok titkosítva van, mivel hitelesítünk a tárolóhoz?
- A Kubernetes Auth megoldja: a ServiceAccount JWT-t **a klaszter biztosítja**, nem mi tároljuk.
- Cloud-on a Workload Identity még tisztább: nincs explicit credential sehol.

Slide: Biztonság és limitációk

- Network dependency: ha a Vault elérhetetlen, új pod-ok nem indulnak (mert nincs Secret). Mitigáció: Vault HA + ESO cache.
- Komplexitás trade-off: minden plusz komponens egy plusz failure mode.
- Az autentikáció maga is titok - ez a secret zero probléma.

Slide: Provider váltás - vendor-függetlenség

- Wow-pont: ugyanaz az `ExternalSecret`, csak a `SecretStore` provider-blokk változik.
- Ha holnap AWS-re költözünk, a Secret-eket nem kell átírni - csak a SecretStore-t.

Slide: Alternatív integrációk

- ESO vs Secrets Store CSI Driver: az ESO **K8s Secret-et generál**, a CSI **közvetlenül volume-ot mountol** a podba (köztes Secret nélkül).
- Vault Agent Sidecar: minden pod mellé sidecar - több erőforrás, de kevesebb függőség az operatorra.
- Sealed Secrets: másik világ - encrypted Secret Git-ben, nem külső tárral.

Slide: DEMO 3

- A wow-pillanat: Vault-ban módosítunk egy értéket → 15 mp múlva a podos log-ban megjelenik az új érték, **restart nélkül**.
- Két terminál szükséges:
  - 1. terminál: `kubectl logs -l app=demo-app -f`
  - 2. terminál: `kubectl exec ... vault kv put ...`
- Ha jut idő: a Vault UI is bemutatható (`port-forward 8200`).